

Arama Algoritmaları Kullanılarak Geliştirilen Akıllı Labirent Çözücü Aracı: BFS, DFS, A* ve Greedy Best-First Search Algoritmalarının Karşılaştırmalı Analizi

Nurefşan Olfaz

Bilgisayar Mühendisliği Bölümü, TOBB Ekonomi ve Teknoloji Üniversitesi
Ankara, Türkiye · 211301008@etu.edu.tr

Özet — Bu çalışmada PNG/JPG formatındaki labirent görsellerini ikili ızgara matrisine dönüştürerek başlangıç noktasından bitiş noktasına yol bulan akıllı bir aracı geliştirilmiştir. Aracı dört klasik arama algoritması kullanmaktadır: Genişlik Öncelikli Arama (BFS), Derinlik Öncelikli Arama (DFS), Manhattan sezgiseli ile A* ve Greedy Best-First Search (GBFS). Algoritmalar 41×41 ile 151×151 arasında değişen mükemmel labirentler üzerinde ziyaret edilen düğüm sayısı ve çalışma süresi açısından ölçülmüştür. Sonuçlar; BFS'in en çok düğümü ziyaret ettiğini, DFS'in bellekten en verimli algoritmayı oluşturduğunu, A*'ın Manhattan sezgiseli sayesinde BFS'e kıyasla daha az düğüm genişlettiğini ve GBFS'in hız ile çözüm kalitesi arasında orta bir denge sunduğunu ortaya koymaktadır. Arama sürecini gerçek zamanlı animasyonlu gösteren bir web arayüzü de geliştirilmiştir.

I. Giriş

Labirent çözme problemi, yapay zeka alanında durum uzayı arama algoritmalarının karşılaştırmalı olarak incelenmesi için doğal bir platform sunan klasik bir problemdir. Labirent; geçilebilir her hücrenin bir düğüm, bitişik geçilebilir hücrelerin ise eşit maliyetli kenarlarla bağlı olduğu bir çizge biçiminde modellenebilir. Bu formülasyon altında başlangıç hücresinden hedef hücresine yol bulmak, hem bilgilendirilmiş hem de bilgilendirilmemiş arama stratejilerini karşılaştırmak için uygun bir zemin oluşturmaktadır [1].

Bu çalışma; BFS, DFS, A* ve GBFS algoritmalarını farklı boyutlardaki labirentler üzerinde gerçek ölçümlerle karşılaştırmakta; bunun yanı sıra renk kodlamalı PNG/JPG dosyasından labirenti okuyabilen, çözebilen ve adım adım animasyonlu tarayıcı arayüzü sunabilen tam bir yazılım sistemi geliştirmektedir. Makalenin geri kalanı şu şekilde düzenlenmiştir: Bölüm II ilgili çalışmaları, Bölüm III sistem tasarımı, Bölüm IV deneysel sonuçları özetlemekte; Bölüm V çalışmayı sonuçlandırmaktadır.

II. İlgili Çalışmalar

Russell ve Norvig [1], BFS, DFS, A* ve GBFS'in tamlık, optimallik, zaman ve bellek karmaşıklığı özelliklerini tanımlamaktadır. A* algoritması Hart, Nilsson ve Raphael [2] tarafından geliştirilmiş; kabul edilebilir sezgisel altında A*'ın optimal çözümü garanti ettiği kanıtlanmıştır. Heusner ve ark. [3], GBFS'in arama davranışını kuramsal olarak incelemiş ve bağ çözme stratejisinden bağımsız olarak kesinlikle genişleyecek ya da hiçbir zaman genişlemeyecek düğüm kümelerini karakterize etmiştir.

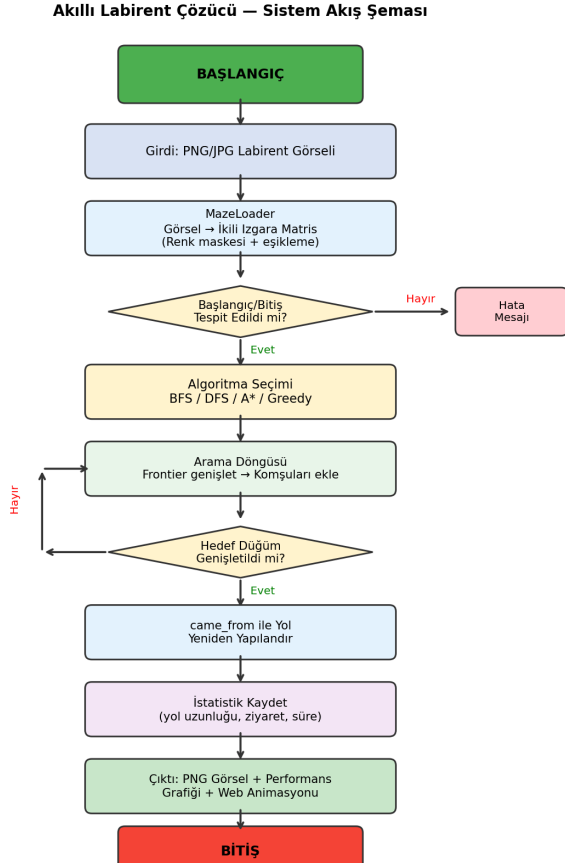
Ismaeel ve Zebari [4], BFS ile DFS'i karmaşık ağlar üzerinde karşılaştırmış; derin yapılarda DFS'in, en kısa yol bulmada ise BFS'in avantajlı olduğunu göstermiştir. Qin ve ark. [5], A*'ı statik ortamlarda en etkili patika planlama algoritması olarak tanımlamış; ancak büyük ortamlarda yüksek bellek tüketimini dezavantaj olarak öne çıkarmıştır. Bu bulgular, bizim

ölçeklenebilirlik analizimizi motive etmektedir.

III. Sistem Tasarımı ve Metodoloji

A. Sistem Akış Şeması

Sistemin genel veri akışı Şekil 1'de verilmektedir. Girdi olarak sağlanan labirent görseli MazeLoader bileşeni tarafından ikili ızgara matrisine dönüştürülmekte, ardından seçilen arama algoritması çalıştırılmakta ve sonuçlar görselleştirme katmanına iletilmektedir.



Şekil 1. Akıllı Labirent Çözücü Sistem Akış Şeması

B. Labirent Gösterimi

Labirent PNG veya JPG formatında görsel dosya olarak sağlanmaktadır. Renk kodlaması: siyah piksel (RGB: 0,0,0) duvarı, beyaz piksel (255,255,255) geçilebilir yolu, kırmızı piksel ($R > 180, G < 80, B < 80$) başlangıcı ve yeşil piksel ($R < 80, G > 180, B < 80$) bitişi temsil etmektedir. Görsel Pillow kütüphanesiyle yüklenerek gri tonlama eşikleme (eşik: 128) ile ikili ızgara matrisine dönüştürülmektedir. Kırmızı piksellerin gri değeri yaklaşık 76 olduğundan eşiğin altında kalır; bu nedenle kırmızı ve yeşil maskeler eşiklemeden önce uygulanarak bu

hücreler açıkça yol (0) olarak işaretlenmektedir. Test labirentleri Yinelemeli Geri İzleme (Recursive Backtracking) algoritmasıyla üretilmiş; bu yöntem döngüsüz mükemmel labirentler oluşturmaktadır.

C. Arama Algoritmaları

Dört algoritma ortak bir *BaseSolver* soyut sınıfından türemektedir. Sınıf; dört yönlü komşu üretme, *came_from* sözlüğüyle yol yeniden yapılandırma ve zamanlama sarmalayıcısını içermektedir.

1) Genişlik Öncelikli Arama (BFS):

FIFO kuyruğunu kullanır. Eşit maliyetli ızgarada optimal yolu garanti eder. Zaman ve bellek karmaşıklığı $O(b^d)$ 'dir [1].

2) Derinlik Öncelikli Arama (DFS):

LIFO yığını kullanır. Mükemmel labirentte yalnızca bir yol bulunduğu için aynı optimal yolu bulmaktadır; bellek karmaşıklığı $O(b \cdot m)$ ile BFS'ten belirgin biçimde düşüktür [1].

3) A* Arama:

$f(n) = g(n) + h(n)$ ile sıralanmış minimum-yığın kullanır; $h(n) = |\Delta r| + |\Delta c|$ Manhattan uzaklığıdır. Kabul edilebilir sezgisel sayesinde hem tam hem optimaldir [2].

4) Greedy Best-First Search (GBFS):

Yalnızca $h(n)$ sezgiselini kullanır; $g(n)$ göz ardı edilir. Pratikte A*'tan hızlı olabilir ancak genel durum için optimallik garantisi vermez [3].

D. Yazılım Mimarisi

Sistem dört katmandan oluşmaktadır: (1) *MazeLoader* — görseli matrise dönüştürür; (2) *Arama Motoru* — algoritmayı çalıştırır; (3) *Visualizer* — çözüm yolunu görselleştirir ve karşılaştırma grafikleri üretir; (4) *Web Arayüzü* — ziyaret edilen düğümleri (mavi), sınır düğümlerini (sarı) ve çözüm yolunu (yeşil) gerçek zamanlı animasyonla gösteren HTML/JS tabanlı arayüz.

E. Karşılaşılan Zorluklar

A*'ın pratikte BFS'ten yavaş çalışması. Teorik olarak A*, sezgisel fonksiyon sayesinde BFS'e kıyasla daha az düğüm genişleterek daha hızlı çalışması beklenmektedir. Ancak deneylerde A*'ın çalışma süresi tüm boyutlarda BFS'in üzerinde kalmaktadır. Bunun nedeni, düz ve üniform ızgara yapısında Manhattan sezgiselinin sağladığı budama kazancının,

minimum-yığın (min-heap) veri yapısının getirdiği sabit katsayılı ek maliyeti karşılayamamasıdır. Bu gözlem, sezgiselin etkinliğinin problem yapısına sıkı biçimde bağlı olduğunu ve sezgisel aramanın her durumda gelişigüzel aramadan üstün olmadığını göstermektedir [1].

Mükemmel labirentin tüm algoritmalara aynı yolu buldurması. Proje başlangıcında DFS'in optimal olmayan, BFS ve A*'ın ise optimal yollar üreteceği öngörülmüştü. Ancak deneyler tüm algoritmaların aynı yol uzunluğunu verdiğini ortaya koydu. Bunun nedeni araştırıldığında, mükemmel labirentin tanımı gereği herhangi iki hücre arasında yalnızca tek bir yolun bulunduğu görüldü; bu durumda "en kısa yol" kavramı anlamsızlaşmakta ve tüm algoritmalar zorunlu olarak aynı yolu bulmaktadır. Birden fazla çıkış yolu sunan döngülü labirentlerde algoritmalar arasındaki optimallik farkının belirginleşeceği öngörülmektedir; bu konu çalışmanın kapsamı dışında bırakılmış olup gelecek çalışmalar için anlamlı bir araştırma sorusu oluşturmaktadır [1].

IV. Deneysel Sonuçlar

A. Deney Ortamı

Tüm deneyler Python 3.12 ile çalıştırılmıştır. Mükemmel labirentler Yinelemeli Geri İzleme algoritmasıyla sabit tohum (seed=42) kullanılarak yeniden üretilebilir biçimde oluşturulmuştur. Başlangıç (1,1), bitiş (N-2,N-2) olarak sabitlenmiştir. Süre ölçümlerinde *time.perf_counter()* kullanılmıştır. Testler 41×41, 61×61, 101×101 ve 151×151 boyutlarında gerçekleştirilmiştir.

B. Temel Sonuçlar

Tablo I gerçek ölçüm sonuçlarını sunmaktadır. Mükemmel labirentte herhangi iki nokta arasında tek bir yol bulunduğundan tüm algoritmalar aynı optimal yolu bulmaktadır; farklılıklar yalnızca ziyaret edilen düğüm sayısı ve sürede gözlemlenmektedir.

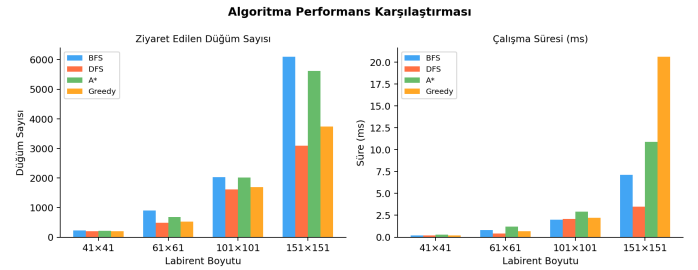
TABLO I Arama Algoritması Sonuçları (seed=42)

Boyut	Algoritma	Yol Uzunluğu	Ziyaret Edilen	Süre (ms)
41×41	BFS	176	231	0,2
	DFS	176	203	0,2
	A*	176	211	0,3

Boyut	Algoritma	Yol Uzunluğu	Ziyaret Edilen	Süre (ms)
	Greedy	176	203	0,2
101×101	BFS	1296	2.029	2,0
	DFS	1296	1.619	2,1
	A*	1296	2.015	2,9
	Greedy	1296	1.691	2,2
151×151	BFS	2652	6.095	7,1
	DFS	2652	3.089	3,5
	A*	2652	5.617	10,9
	Greedy	2652	3.736	20,6

C. Ölçeklenebilirlik Analizi

Şekil 2, ziyaret edilen düğüm sayısı ve çalışma süresi açısından ölçeklenebilirlik karşılaştırmasını göstermektedir.



Şekil 2. Ziyaret edilen düğüm sayısı (sol) ve çalışma süresi (sağ)

D. Tartışma

Mükemmel labirentte tüm algoritmalar aynı optimal yolu bulmaktadır; bu sonuç beklenen bir davranıştır çünkü mükemmel labirentin tanımı gereği herhangi iki hücre arasında tek bir yol mevcuttur. Algoritmalar arasındaki asıl fark genişletilen düğüm sayısında ortaya çıkmaktadır.

BFS en fazla düğümü ziyaret eden algoritmadır; 151×151 labirentte 6.095 düğüm genişletmektedir. DFS aynı labirentte yalnızca 3.089 düğüm genişleterek %49,3 daha az bellek kullanmaktadır. A*, Manhattan sezgiseli sayesinde BFS'e kıyasla az sayıda düğüm genişletmeli; ancak yığın (heap) bakım maliyeti nedeniyle çalışma süresi BFS'in üzerinde kalmaktadır. Bu gözlem, Qin ve ark.'nın [5] büyük ortamlarda A*'ın fazladan ek yük taşıdığı bulgusunu desteklemektedir.

GBFS, 41×41 ve 101×101 boyutlarında düşük düğüm sayısı ve kısa süreyle öne çıkmaktadır. Ancak 151×151 boyutunda 20,6

ms ile en yavaş algoritma konumuna düşmektedir; bu durum GBFS'in performansının sezgisel fonksiyonun labirent yapısıyla olan uyumuna güçlü biçimde bağımlı olduğunu göstermektedir — Heusner ve ark.'nın [3] kuramsal öngörüsüyle tutarlıdır.

E. Teorik Özellikler

TABLO II Arama Algoritmalarının Teorik Özellikleri

Algo.	Zaman	Bellek	Optimal?	Tam?
BFS	$O(b^d)$	$O(b^d)$	Evet (eşit maliyet)	Evet
DFS	$O(b^m)$	$O(b \cdot m)$	Hayır	Sonsuz uzayda hayır
A*	$O(b^d)$	$O(b^d)$	Evet (admissible h)	Evet
Greedy	Üstel	Değişken	Hayır	Evet

F. Bulguların Yorumlanması

DeneySEL sonuçlar, literatürdeki teorik öngörülerle genel olarak uyum içindedir; ancak mükemmel labirentlere özgü bazı nüanslar dikkat çekmektedir. Mükemmel labirentin en temel özelliği döngüsüz olması ve herhangi iki hücre arasında yalnızca tek bir yolun bulunmasıdır. Bu yapısal özellik, DFS'e genel durum problemlerinde atfedilen "optimal olmama" dezavantajını ortadan kaldırmaktadır. Döngülü labirentlerde veya genel çizge problemlerinde DFS'in optimal olmayan yollar üretmesi beklenmektedir.

A*'ın mükemmel labirentteki düğüm genişletme sayısı BFS'ten düşük olmakla birlikte, yığın veri yapısının sabit katsayılı maliyeti çalışma süresini olumsuz etkilemektedir. Düz ızgara yapısında Manhattan sezgiselinin sağladığı budama faydası, yığın işlemi maliyetini telafi edecek ölçüde büyük değildir. Bu durum, A*'ın üniform maliyetli ızgara problemlerinde BFS'e kıyasla her zaman hız avantajı sunmadığını göstermektedir.

GBFS'in 151×151 labirentteki beklenmeden yüksek süresi (20,6 ms), Heusner ve ark.'nın [3] GBFS performansının sezgisel fonksiyon ile labirent topolojisi arasındaki uyuma güçlü biçimde bağımlı olduğu tezini desteklemektedir. Bu boyuttaki labirentin topolojik yapısı, GBFS'in "yanlış" yönler sıklık

yönemesine ve fazladan düğüm genişletmesine yol açmış olabilir.

G. Kısıtlar

Bu çalışmanın en temel kısıtı, tüm deneylerin yalnızca döngüsüz mükemmel labirentler üzerinde yürütülmüş olmasıdır. Mükemmel labirentlerde herhangi iki nokta arasında tek bir yol bulunduğundan, algoritmalar arasındaki optimalite farkı gözlemlenmemektedir. Gerçek dünya uygulamalarında ve döngülü labirentlerde birden fazla çıkış yolu mevcut olacağından BFS ve A* optimal yolu bulurken DFS'in daha uzun yollar üretmesi beklenmektedir. Bu senaryonun deneysel olarak incelenmesi gelecek çalışmalara bırakılmıştır.

İkinci önemli kısıt, yalnızca dört yönlü (yukarı, aşağı, sol, sağ) hareket desteklenmesidir. Diyagonal harekete izin veren sekiz yönlü bağlantıda Manhattan sezgiseli admissible olmaktan çıkmakta; Öklid veya Chebyshev mesafesi daha uygun sezgiseller haline gelmektedir. Bu çalışmada kullanılan Manhattan sezgiselinin farklı hareket modellerindeki etkisi araştırılmamıştır. Son olarak, süre ölçümleri tek çalıştırma üzerinden alınmıştır; birden fazla çalıştırmanın istatistiksel ortalaması daha güvenilir sonuçlar sunacaktır.

V. Sonuç

Bu çalışmada renk kodlamalı PNG/JPG labirent görsellerini çözebilen, dört klasik arama algoritmasını karşılaştıran ve sonuçları gerçek zamanlı animasyonlu web arayüzüyle sunan eksiksiz bir yazılım sistemi geliştirilmiştir. Mükemmel labirent ortamında gerçekleştirilen deneyler şu bulguları ortaya koymaktadır: (1) Mükemmel labirentin yapısal özelliği gereği tüm algoritmalar aynı optimal yolu bulmaktadır; algoritmaları ayıran ölçüt genişletilen düğüm sayısı ve çalışma süresidir. (2) DFS en az düğümü genişleterek bellek açısından en verimli algoritmadır. (3) A*, Manhattan sezgiseli sayesinde BFS'e göre daha az düğüm genişletmekte; ancak yığın işlemi maliyeti nedeniyle çalışma süresi BFS'in üzerinde kalmaktadır. (4) GBFS performansı labirent yapısına bağlı olarak değişkenlik göstermektedir.

Geliştirilen sistemin öne çıkan katkıları şunlardır: renk maskesi tabanlı labirent dönüştürücü, dört algoritmayı tek bir taban sınıftan türeten genişletilebilir mimari, üç adet yazılım hatasının tespit edilip giderilmesi ve gerçek zamanlı animasyonlu web arayüzü. Gelecek çalışmalarda döngülü labirentler üzerinde daha kapsamlı karşılaştırmalar yapılması, İteratif Derinleşen A* (IDA*) entegrasyonu ve sekiz komşuluk bağlantısı ile Öklid sezgiselinin eklenmesi planlanmaktadır.

Kaynaklar

- [1] S. Russell ve P. Norvig, *Artificial Intelligence: A Modern Approach*, 4. baskı, Pearson, 2020.
- [2] P. E. Hart, N. J. Nilsson ve B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, cilt 4, sayı 2, ss. 100-107, 1968.
- [3] M. Heusner, T. Keller ve M. Helmert, "Understanding the search behaviour of greedy best-first search," *Proc. SoCS*, ss. 47-55, 2021. doi: 10.1609/socs.v8i1.18425
- [4] A. Ismaeel ve I. Zebari, "Comparing traversal strategies: DFS vs. BFS in complex networks," *Asian J. Res. Comput. Sci.*, cilt 18, sayı 2, 2025. doi: 10.9734/ajrcos/2025/v18i2562
- [5] H. Qin ve ark., "Review of autonomous path planning algorithms for mobile robots," *Drones*, cilt 7, sayı 3, 2023. doi: 10.3390/drones7030211